The Atom API

Table of Contents

1 A Brief History of Weblogging APIs 1
1.1 LiveJournal API 1
1.2 Example: LiveJournal login 1
1.3 LiveJournal goes XML−RPC 1
1.4 ManilaRPC 1
1.5 Blogger API 1
1.6 Blogger API limitations 2
1.7 Metaweblog API 2
1.8 Metaweblog API limitations 2
1.9 Example: creating a new entry with Metaweblog API 2
1.10 Example: Metaweblog authentication 3
1.11 Metaweblog "extensibility" 3
1.12 Example: specifying an entry source 4
1.13 The category problem 4
1.14 Example: an multiple categories 4
1.15 Metaweblog "extensibility": namespaces 5
1.16 All my life's a circle 5

2 The Atom API 6
2.1 Guiding Principles 6
2.2 Technical architecture 6
2.3 API Discovery 6
2.4 API Auto−discovery 6
2.5 Example: Home page with Atom API auto−discovery link 6
2.6 Introspection file 7
2.7 Example: Atom introspection file 7
2.8 Finding entries 7
2.9 Example: Getting recent entries 7
2.10 Creating, retrieving, editing, deleting 8
2.11 URIs 8
2.12 Example: Retrieving an existing entry 8
2.13 Example: Creating a new entry 9
2.14 Example: Extensibility through namespaces 9
2.15 Example: Modifying an existing entry 9
2.16 Example: Deleting an entry 10
2.17 Other features defined by the Atom API 10

3 What the Atom API isn't 11
3.1 It isn't SOAP 11
3.2 It isn't RDF 11
3.3 It isn't WebDAV 11
3.4 Further reading 11

1 A Brief History of Weblogging APIs

1.1 LiveJournal API

- LiveJournal is a GPL weblog/community system
- API started as "flat" HTTP−based client−server protocol
- http://livejournal.com/doc/server/ljp.csp.flat.protocol.html
- Functions were called by sending HTTP POST with form−encoded parameters
- Values were returned in plain text, key−value pairs separated by carriage returns
- API developed by LiveJournal, specific to LiveJournal, never adopted by anyone else

1.2 Example: LiveJournal login

```
POST /interface/flat HTTP/1.1
Host: www.livejournal.com
Content−type: application/x−www−form−urlencoded

mode=login&user=test&password=test

HTTP/1.1 200 OK
Content−Type: text/plain

name
Mr. Test Account
success
OK
message
Hello Test Account!
```

1.3 LiveJournal goes XML−RPC

- LiveJournal later added an XML−RPC interface
- http://livejournal.com/doc/server/ljp.csp.xml−rpc.protocol.html
- Main reasons: perceived popularity of XML−RPC, ease of marshalling/unmarshalling
- XML was cool, and XML−RPC was tirelessly advocated until all major languages supported it

1.4 ManilaRPC

- Manila is a content management system developed by UserLand Software
- http://xmlrpc.com/manilaRpc
- Comprehensive API, supports everything you can do through Manila's web interface
- ManilaRPC supports both XML−RPC and RPC−style SOAP
- API developed by UserLand, specific to Manila, never adopted by anyone else

1.5 Blogger API

- Blogger is a weblog publishing tool by Pyra Labs, now owned by Google
- http://www.blogger.com/developers/api/1_docs/
- API is XML−RPC only

- Limited functionality: `newPost`, `editPost`, `getUsersBlogs`, `getUserInfo`, `getTemplate`, `setTemplate`
- Created by Evan Williams as an experiment, immediately adopted by UserLand, and later by SixApart and many others
- Now the most widely adopted weblogging web service

1.6 Blogger API limitations

- Limited functionality
- RPC−style function calls not extensible
- Wire format is complicated
- Passes authentication credentials over the wire in plaintext

1.7 Metaweblog API

- Developed by UserLand in response to perceived weaknesses of Blogger API
- http://www.xmlrpc.com/metaWeblogApi
- "Solves" extensibility problem by using structs for all inputs and outputs
- "Solves" functionality problem by defining new methods and documenting new parameters (within The Struct) for existing methods
- Supported by some other vendors

1.8 Metaweblog API limitations

- Extensibility is actually more difficult than "put everything in a struct"
- Half an API; meant to be implemented alongside Blogger API
- Wire format is insanely complicated
- Passes authentication credentials over the wire in plaintext

1.9 Example: creating a new entry with Metaweblog API

```
POST /mt/mt−xmlrpc.cgi HTTP/1.1
Host: example.com
Content−Type: text/xml

<?xml version='1.0'?>
<methodCall>
  <methodName>metaWeblog.newPost</methodName>
  <params>
    <param>
      <value>
        <string>BLOGID</string>
      </value>
    </param>
    <param>
      <value>
        <string>USERNAME</string>
      </value>
    </param>
    <param>
      <value>
        <string>PASSWORD</string>
      </value>
```

```
    </param>
    <param>
      <value>
        <struct>
          <member>
            <name>dateCreated</name>
            <value>
              <dateTime.iso8601>ENTRY_DATE</dateTime.iso8601>
            </value>
          </member>
          <member>
            <name>description</name>
            <value>
              <string>ENTRY_DESCRIPTION</string>
            </value>
          </member>
          <member>
            <name>title</name>
            <value>
              <string>ENTRY_TITLE</string>
            </value>
          </member>
        </struct>
      </value>
    </param>
    <param>
      <value>
        <int>1</int>
      </value>
    </param>
  </params>
</methodCall>
```

1.10 Example: Metaweblog authentication

From the wire trace:

```
   <param>
      <value>
        <string>USERNAME</string>
      </value>
    </param>
    <param>
      <value>
        <string>PASSWORD</string>
      </value>
    </param>
```

Cleartext passwords! Party like it's 1979!

1.11 Metaweblog "extensibility"

From the spec:

1.11.1 Transmitting elements with attributes

The members of the struct passed in newPost and editPost come from the elements of items in RSS 2.0. The most commonly used core elements have no attributes, so it's clear how to include them in the struct. However, some elements, such as source, enclosure and category, may have attributes and a value. Here are a simple set of rules for elements that have attributes and a value. Note that these rules do not apply to enclosure and source, which are provided for specifically above.

1. If an element has attributes, then represent the element with a struct, and include the attributes as sub−elements of the struct.
2. If an element has both attributes and a value, make the element a struct, include the attributes as sub−elements, and create a sub−element for the value with the name _value. Note that this means that no element can be passed through the API that has an attribute whose name is _value.

1.12 Example: specifying an entry source

Add this member to The Struct:

```
<member>
   <name>source</name>
   <value>
      <struct>
        <member>
          <name>url</name>
          <value>http://partners.userland.com/nyt/nytHomepage.xml</value>
        </member>
        <member>
          <name>_value</name>
          <value>New York Times: NYT HomePage</value>
        </member>
      </struct>
   </value>
 </member>
```

1.13 The category problem

- Entries can be filed in multiple categories
- But since we're using a struct with "category" as the key, multiple categories can't be represented (duplicate keys)
- Instead of acknowledging this problem and solving the general case, UserLand created a workaround specifically for categories

1.14 Example: an multiple categories

```
<member>
   <name>category</name>
   <value>
      <array>
        <data>
          <value>RSS</value>
          <value>XMLRPC</value>
```

```
        </data>
      </array>
    </value>
  </member>
```

1.15 Metaweblog "extensibility": namespaces

From the spec:

1.15.1 Transmitting elements from namespaces

RSS 2.0 allows for the use of namespaces. If you wish to transmit an element that is part of a namespace include a sub−struct in the struct passed to newPost and editPost whose name is the URL that specifies the namespace. The sub−element(s) of the struct are the value(s) from the namespace that you wish to transmit.

1.16 All my life's a circle

We have now come full circle: we've managed to reinvent XML, over RPC, over XML... badly

More discussion at http://www.intertwingly.net/stories/2003/01/26/evolve.html

2 The Atom API

2.1 Guiding Principles

- 100% vendor−neutral
- Implemented by everybody
- Freely and easily extensible by anybody
- Cleanly and thoroughly specified
- Secure

2.2 Technical architecture

- REST, not XML−RPC or SOAP
- Doc−literal, not RPC
- Use well−defined vocabulary for specific purpose (not trying to reinvent WebDAV)
- Take full advantage of XML (and namespaces)
- Take full advantage of native transport (currently HTTP)
- No cleartext passwords (duh)

2.3 API Discovery

Previous weblog services:

- No standard for API discovery
- Left it up to end user to provide exact API URL (`http://example.com/mt/mt−xmlrpc.cgi`)
- Some servers implemented undocumented functions (`deletePost`)
- Client software had to guess what functionality was provided and what extensions were supported

Goals for Atom:

- Fix all of that

2.4 API Auto−discovery

- Assume end user knows their home page
- `LINK` tag in `HEAD` of home page points to Atom introspection file
- Atom introspection file lists supported functions and extensions

2.5 Example: Home page with Atom API auto−discovery link

```
<!DOCTYPE html PUBLIC "−//W3C//DTD XHTML 1.0 Strict//EN"
  "http://www.w3.org/TR/xhtml1/DTD/xhtml1−strict.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" lang="en" xml:lang="en">
<head>
<title>My Weblog</title>
<link rel="service.edit" type="application/x.atom+xml"
  href="/myblog/atom.cgi/introspection" title="Atom API" />
```

```
</head>
<body>
...
</body>
</html>
```

2.6 Introspection file

- Lists supported functions and extensions
- Simple, well−defined XML format
- Vendors can extend introspection file with XML namespaces

2.7 Example: Atom introspection file

```
GET /myblog/atom.cgi/introspection HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
Content−Type: application/x.atom+xml

<?xml version="1.0" encoding="utf−8"?>
<introspection xmlns="http://purl.org/atom/ns#" >
  <create−entry>http://example.com/myblog/atom.cgi/edit</create−entry>
  <search−entries>http://example.com/myblog/atom.cgi/search</search−entries>
  <edit−template>http://example.com/atom.cgi/templates</edit−template>
  <user−prefs>http://example.com/myblog/atom.cgi/prefs</user−prefs>
  <categories>http://example.com/atom.cgi/categories</categories>
</introspection>
```

2.8 Finding entries

- Use `search−entries` URI specified in introspection file
- Add query string parameters such as `atom−last` (recent entries)
- More complex examples defined in Atom API spec

2.9 Example: Getting recent entries

```
GET /myblog/atom.cgi/search?atom−last=20 HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
Content−Type: application/x.atom+xml

<search−results xmlns="http://purl.org/atom/ns#" >
  <entry>
    <title>My Second Post</title>
    <id>http://example.com/atom.cgi/edit/2</id>
  </entry>
  <entry>
    <title>My First Post</title>
    <id>http://example.com/atom.cgi/edit/1</id>
  </entry>
</search−results>
```

2.10 Creating, retrieving, editing, deleting

Take full advantage of HTTP:

- Creating a new entry is an HTTP POST to the `create-entry` URI specified in the introspection file
- Retrieving an entry is an HTTP GET on the entry's URI (returned after creating new entry, or in search results)
- Editing an entry is an HTTP PUT on the entry's URI
- Deleting an entry is an HTTP DELETE on the entry's URI

2.11 URIs

- "The entry's URI" is implementation-specific
- These examples route everything through a single script (`atom.cgi`), just to prove that you can do that
- You don't have to do that
- You could specify that you create new entries by POSTing to the home page, and retrieve/edit/delete through the same URL as you would view the entry as HTML in a web browser
- Introspection file rules all

2.12 Example: Retrieving an existing entry

```
GET /myblog/atom.cgi/edit/1 HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
Content-Type: application/x.atom+xml

<?xml version="1.0" encoding="utf-8"?>
<entry xmlns="http://purl.org/atom/ns#">
  <title>My First Entry</title>
  <summary>My First Entry Excerpt (generally plaintext)</summary>
  <author>
    <name>Bob B. Bobbington</name>
    <email>bob@example.com</email>
    <url>http://homepage.example.com/</url>
  </author>
  <issued>2003-11-19T02:29:29</issued>
  <created>2003-11-19T04:10:58Z</created>
  <modified>2003-11-19T04:22:03Z</modified>
  <link>http://example.com/myblog/archives/2003/11/19/My_First_Entry.html</link>
  <id>urn:example-com:myblog:1</id>
  <content type="application/xhtml+xml" xml:lang="en">
    <div xmlns="http://www.w3.org/1999/xhtml">
      <p>Hello, <em>weblog</em> world!</p>
      <p>This is my first post <strong>ever</strong>!</p>
    </div>
    </content>
</entry>
```

2.13 Example: Creating a new entry

```
POST /myblog/atom.cgi/edit HTTP/1.1
Host: example.com
Content-Type: application/x.atom+xml

<?xml version="1.0" encoding="utf-8"?>
<entry xmlns="http://purl.org/atom/ns#">
  <title>My Entry Title</title>
  <created>2003-11-17T12:29:29Z</created>
  <content type="application/xhtml+xml" xml:lang="en">
    <div xmlns="http://www.w3.org/1999/xhtml">
      <p>Hello, <em>weblog</em> world!</p>
      <p>This is my third post <strong>ever</strong>!</p>
    </div>
  </content>
</entry>

HTTP/1.1 201 Created
Location: http://example.com/myblog/atom.cgi/edit/3
```

2.14 Example: Extensibility through namespaces

Take full advantage of XML:

```
POST /myblog/atom.cgi HTTP/1.1
Host: example.com
Content-Type: application/x.atom+xml

<?xml version="1.0" encoding="utf-8"?>
<entry
  xmlns="http://purl.org/atom/ns#"
  xmlns:mt="http://www.movabletype.org/atom/ns#">

  <title>My Entry Title</title>
  <created>2003-11-17T12:29:29Z</created>
  <mt:allowComments>1</mt:allowComments>
  <content type="application/xhtml+xml" xml:lang="en">
    <div xmlns="http://www.w3.org/1999/xhtml">
      <p>Hello, <em>weblog</em> world!</p>
      <p>This is my first post <strong>ever</strong>!</p>
    </div>
  </content>
</entry>
```

2.15 Example: Modifying an existing entry

```
PUT /myblog/atom.cgi/edit/1 HTTP/1.1
Host: example.com
Content-Type: application/x.atom+xml

<?xml version="1.0" encoding="utf-8"?>
<entry xmlns="http://purl.org/atom/ns#">
  <title>My First Entry</title>
  <summary>My First Entry Excerpt (generally plaintext)</summary>
  <author>
    <name>Bob B. Bobbington</name>
```

```
    <email>bob@example.com</email>
    <url>http://homepage.example.com/</url>
  </author>
  <issued>2003−11−19T02:29:29</issued>
  <created>2003−11−19T04:10:58Z</created>
  <modified>2003−11−19T04:22:03Z</modified>
  <link>http://example.com/myblog/archives/2003/11/19/My_First_Entry.html</link>
  <id>urn:example−com:myblog:1</id>
  <content type="application/xhtml+xml" xml:lang="en">
    <div xmlns="http://www.w3.org/1999/xhtml">
      <p>Hello, <em>weblog</em> world!</p>
      <p>This is my first post <strong>ever</strong>!</p>
    </div>
    </content>
</entry>

HTTP/1.1 205 Reset Content
```

2.16 Example: Deleting an entry

```
DELETE /myblog/atom.cgi/edit/3 HTTP/1.1
Host: example.com

HTTP/1.1 200 OK
```

2.17 Other features defined by the Atom API

- Posting comments
- Managing users
- Managing user preferences
- Managing site templates
- Managing categories
- http://bitworking.org/rfc/draft−gregorio−07.html

3 What the Atom API isn't

3.1 It isn't SOAP

- Atom format is already an XML wrapper around entry content
- Don't need a wrapper around the wrapper
- Zero vendor interest in transport independence
- Use HTTP authentication instead of WS−Security

3.2 It isn't RDF

- Simple, well−defined XML format
- A vocal minority would like this project (and every XML−centric project) to use RDF instead
- None of these people actually write software
- Compromise: normative transform from Atom XML to squeaky−clean RDF

3.3 It isn't WebDAV

- Atom is designed specifically for editing episodic web sites
- Atom provides mediated access to multiple vendor−specific backends via a common API
- Atom must be implementable by independent site maintainers on hosted accounts (CGI, no root access)
- Limited scope, specific audience
- Not all things to all people

3.4 Further reading

- http://bitworking.org/rfc/draft−gregorio−07.html
- http://intertwingly.net/wiki/pie/
- http://www.imc.org/atom−syntax/

This presenation has ended.
Click anywhere to return.

_

